Exercice 0

Créer un **script PowerShell** qui génère une structure de dossiers pour un **projet informatique**.

Exigences :

1. Créer **10 dossiers principaux** nommés :

```
Projet01
Projet02
...
Projet10
```

2. Dans chaque dossier **ProjetXX**, créer les sous-dossiers :

```
Documentation
Code
Tests
```

3. Dans le dossier **Documentation**, créer un fichier :

```
README.md
```

4. Le script doit :

- afficher des **messages de progression**
- utiliser **Join-Path** pour les chemins
- formater les numéros avec **deux chiffres**
- utiliser **-Force** et **Out-Null**

Correction

```
# Script PowerShell pour créer une structure de projets
# Etudiants: Nom1 Nom2

# On récupère le dossier courant
$cheminRacine = Get-Location

# Boucle pour créer 10 projets
foreach ($i in 1..10) {

    # Conversion du numéro en deux chiffres
    $numeroProjet = $i.ToString("00")
```

```
    # Nom du dossier projet
    $dossierProjet = "Projet" + $numeroProjet

    # Construction du chemin complet
    $cheminProjet = Join-Path $cheminRacine $dossierProjet

    Write-Host "Création du dossier $dossierProjet"

    # Création du dossier principal
    New-Item -Path $cheminProjet -ItemType Directory -Force | Out-Null

    # Création des sous-dossiers
    $cheminDoc = Join-Path $cheminProjet "Documentation"
    $cheminCode = Join-Path $cheminProjet "Code"
    $cheminTests = Join-Path $cheminProjet "Tests"

    New-Item -Path $cheminDoc -ItemType Directory -Force | Out-Null
    New-Item -Path $cheminCode -ItemType Directory -Force | Out-Null
    New-Item -Path $cheminTests -ItemType Directory -Force | Out-Null

    Write-Host " Dossiers Documentation, Code et Tests créés"

    # Création du fichier README.md
    $cheminReadme = Join-Path $cheminDoc "README.md"
    New-Item -Path $cheminReadme -ItemType File -Force | Out-Null

    Write-Host " Le fichier README.md a été créé dans Documentation"
}

Write-Host "Création de la structure de projets terminée."
```

Exercice 1 — Date utilisateur

Énoncé

Écrire un script PowerShell qui :

1. Demande à l'utilisateur d'entrer une **date**.
2. Si la date ne peut pas être convertie correctement, la demande doit **se répéter dans une boucle**.
3. Calculer le **temps restant jusqu'à cette date**.
4. Afficher le temps restant en **jours, heures, minutes et secondes**.

Solution

```
$valide = $false

while ($valide -eq $false) {

    $dateInput = Read-Host "Entrer une date"

    $date = $dateInput -as [datetime]

    if ($date -ne $null) {
        $valide = $true
    }
}

$maintenant = Get-Date
$tempsRestant = $date - $maintenant

Write-Host "$($tempsRestant.Days) jours"
Write-Host "$($tempsRestant.Hours) heures"
Write-Host "$($tempsRestant.Minutes) minutes"
Write-Host "$($tempsRestant.Seconds) secondes"
```

Exercice 2 — Ping parc informatique

Énoncé

Créer un script `.ps1` qui :

1. Lit un **fichier CSV contenant une colonne IP**.
2. Envoie **un seul ping par adresse**.
3. Si le ping fonctionne :
   - récupérer le **nom DNS**
   - noter **En ligne**
4. Sinon :
   - noter **Injoignable**
5. Sauvegarder les résultats dans **Resultats.csv**.

Solution

```
$machines = Import-Csv "machines.csv"

$resultats = @()

foreach ($machine in $machines) {
```

```
    $ip = $machine.IP

    $ping = Test-Connection $ip -Count 1 -Quiet

    if ($ping) {
        $nom = ([System.Net.Dns]::GetHostEntry($ip)).HostName
        $statut = "En ligne"
    }
    else {
        $nom = "Inconnu"
        $statut = "Injoignable"
    }

    $objet = [PSCustomObject]@{
        AdresseIP = $ip
        NomServeur = $nom
        StatutDernierCheck = $statut
    }

    $resultats += $objet
}

$resultats | Export-Csv "Resultats.csv" -NoTypeInformation
```

Exercice 3 — Nombre premier

Énoncé

Écrire un script qui :

1. Trouve les **n premiers nombres premiers**.
2. Le nombre est passé en **paramètre**.
3. Valeur par défaut **100**.
4. Le paramètre :
   - position **0**
   - entre **1 et 500**.

Solution

```
param(
    [Parameter(Position=0)]
    [ValidateRange(1,500)]
    [int]$nbPremiers = 100
)
```

```
$compteur = 0
$nombre = 2

while ($compteur -lt $nbPremiers) {

    $premier = $true

    for ($i=2; $i -lt $nombre; $i++) {

        if ($nombre % $i -eq 0) {
            $premier = $false
        }
    }

    if ($premier) {
        Write-Host $nombre
        $compteur++
    }

    $nombre++
}
```

Exercice 4 — Supprimer le fichier txt le plus vieux

Énoncé

Créer **une seule commande PowerShell** qui :

1. Trouve le fichier **.txt**
2. le plus vieux
3. dans le dossier **Téléchargements**
4. le supprime.

Contraintes :

- utiliser **pipeline**
- ne pas écrire un chemin utilisateur fixe
- utiliser **-WhatIf**.

Solution

```
Get-ChildItem "$HOME\Downloads" -Filter *.txt | Sort-Object LastWriteTime | Select-Object -First 1 | Remove-Item -WhatIf
```

Exercice 5 — Fonction service

Énoncé

Créer une fonction **ModifierService** qui :

1. reçoit :
   - un **nom de service**
   - un **état** : fermer ou redémarrer
2. applique l'état au service.

Ensuite :

- lire un fichier `.txt` contenant des noms de services
- redémarrer chaque service.

Liste :

```
Spooler
Explorer
Wuauserv
Dnscache
WinRM
```

Solution

```
function ModifierService {

    param(
        [string]$nomService,
        [string]$etat
    )

    if ($etat -eq "fermer") {
        Stop-Service $nomService -WhatIf
    }

    if ($etat -eq "redemarrer") {
        Restart-Service $nomService -WhatIf
    }
}

$services = Get-Content "services.txt"

foreach ($service in $services) {
```

```
    ModifierService $service "redemarrer"
}
```

Exercice 6 — Classe voiture

Énoncé

Créer une **classe Voiture** contenant :

- numéro de série
- année
- marque
- modèle.

Créer ensuite **une voiture** avec ces paramètres.

Solution

```
class Voiture {

    [string]$NumeroSerie
    [int]$Annee
    [string]$Marque
    [string]$Modele

    Voiture($serie,$annee,$marque,$modele) {

        $this.NumeroSerie = $serie
        $this.Annee = $annee
        $this.Marque = $marque
        $this.Modele = $modele
    }
}

$v = [Voiture]::new("12345",2024,"Toyota","Corolla")

$v
```

Exercice 7 — Classe avec héritage

Énoncé

Créer :

- classe **Personne**

- classe **Employe** (hérite de Personne)
- classe **Etudiant** (hérite de Personne)

Créer **2 employés et 2 étudiants** et les placer dans une liste.

Solution

```
class Personne {

    [string]$Nom

    Personne($nom) {
        $this.Nom = $nom
    }

    ChangerNom($nom) {
        $this.Nom = $nom
    }
}

class Employe : Personne {

    [datetime]$DateEmbauche
    [double]$TauxHoraire

    Employe($nom,$date,$taux) : base($nom) {
        $this.DateEmbauche = $date
        $this.TauxHoraire = $taux
    }

    ChangerTaux($taux) {
        $this.TauxHoraire = $taux
    }
}

class Etudiant : Personne {

    [datetime]$DateEntree
    [string]$Programme

    Etudiant($nom,$date,$prog) : base($nom) {
        $this.DateEntree = $date
        $this.Programme = $prog
    }

    ChangerProgramme($prog) {
        $this.Programme = $prog
```

```
    }
}

$e1 = [Employe]::new("Jean",(Get-Date),25)
$e2 = [Employe]::new("Marie",(Get-Date),30)

$s1 = [Etudiant]::new("Paul",(Get-Date),"Informatique")
$s2 = [Etudiant]::new("Julie",(Get-Date),"Réseau")

$liste = @($e1,$e2,$s1,$s2)

$liste
```

Exercice 8 — Calculer espace disque

Énoncé

Créer une fonction **CalculerEspace** qui :

1. calcule le **% utilisé du disque C**
2. arrondit aux unités
3. affiche :

- plus de **50% utilisé**
- moins de **50% utilisé**.

Solution

```
function CalculerEspace {

    $disque = Get-CimInstance Win32_LogicalDisk -Filter "DeviceID='C:'"

    $total = $disque.Size
    $libre = $disque.FreeSpace

    $utilise = $total - $libre

    $pourcentage = [math]::Round(($utilise / $total) * 100)

    if ($pourcentage -gt 50) {

        Write-Host "Attention l'espace utilisé est plus de 50% ($pourcentage%) sur
le disque C"
    }
    else {
```

```
        Write-Host "Vous utilisez moins de 50% ($pourcentage%) de l’espace sur le
disque C"
    }
}

CalculerEspace
```